第 4 章　维特比 1967：卷积码的差错界与渐近最优译码

文献档案

题目	Error Bounds for Convolutional Codes and an Asymptotically Optimum Decoding Algorithm（卷积码的差错界与一种渐近最优译码算法）
作者	Andrew J. Viterbi
出处	*IEEE Transactions on Information Theory*, vol. IT-13, no. 2, pp. 260–269, April 1967.
DOI	10.1109/TIT.1967.1054010
原文归档	原文 PDF 未获取（出版社付费墙）。本章据公开摘要与作者后续著作译述。

版本说明：本章依据公开摘要以及 Viterbi 与 Omura 的后续教科书《Principles of Digital Communication and Coding》（1979）译述，**未逐节对照 1967 年原文**。凡原文的定理编号、图表编号与具体曲线数值，本章一律不作精确引用；公式采用后来教科书的标准形式，并在出现处注明。

4.1　摘要与核心命题

论文摘要据公开元数据（OpenAlex 重建）可以译述如下：

> 在无记忆信道上对最优卷积码进行译码时，差错概率可以表示为码的约束长度的函数，并给出上界与下界。除病态信道外，当码率高于 R_0（顺序译码的计算截止速率）时，这些界在渐近意义下（指数阶）是紧的。以约束长度为自变量，最优卷积码的性能优于同等长度的分组码，且这一相对优势随码率增大而增大。上界是针对一种具体的概率型非顺序译码算法得到的；该算法在码率高于 R_0 时被证明是渐近最优的。

这段摘要信息密度很高，可以拆成五条核心命题：

1. **差错界**：以约束长度 K 为参数，给出最优卷积码在无记忆信道上的误码率上界与下界；
2. **阈值现象**：当码率 $R > R_0$ 时，上下界在指数阶上重合；R_0 是顺序译码（sequential decoding）的计算截止速率；
3. **算法**：提出一种固定复杂度的非顺序译码算法，后来以作者之名被称为维特比算法（Viterbi algorithm）；
4. **渐近最优性**：在 $R > R_0$ 时，该算法的差错指数达到下界，因而随着约束长度增大而渐近最优；

5. **与分组码的比较**：在相同长度下，卷积码优于分组码，且优势随码率增大。

这篇论文的分量，不在于某一个孤立的公式，而在于它同时完成了三件事：把卷积码的差错概率写成可计算的界，给出了达到这个界的译码算法，并说清楚了算法在什么码率区间内是“够好”的。此后半个多世纪，卷积码的工程实践基本都绕着这三件事展开。

4.2　卷积码：约束长度与网格表示

4.2.1　编码器与约束长度

设时刻 t 进入编码器的信息比特为 $\boldsymbol{u}_t$，其分量为 $u_t^{(1)},\dots,u_t^{(k)}$；输出码元为 $\boldsymbol{x}_t$，其分量为 $x_t^{(1)},\dots,x_t^{(n)}$，码率 $R=k/n$。卷积码与分组码的根本区别是：输出不仅依赖当前输入，还依赖此前 ν 个输入。为记号简洁，先设 $k=1$（码率 $1/n$），此时 u_t 是单个比特；多输入情形可按分量逐路推广。以下除特别说明外均采用这一简化。对第 j 路输出，

$$x_t^{(j)} = \sum_{i=0}^{\nu} g_i^{(j)}\, u_{t-i} \pmod 2, \qquad j=1,\dots,n, \tag{4.1}$$

其中 $g_i^{(j)}\in\{0,1\}$ 是生成多项式的系数，并约定 $u_{t-i}=0$（$i>t$）。ν 称为记忆阶数（memory order），约束长度通常记为

$$K = \nu + 1. \tag{4.2}$$

不同文献对 K 的定义相差 1：有的把 K 直接定义为移位寄存器的级数（即 ν），有的定义为输出所依赖的输入比特数（即 $\nu+1$）。读旧文献时务必先确认口径，否则“$K=7$ 的码”可能指两种不同的寄存器长度。

编码器的状态是最近 ν 个输入比特：

$$s_t = (u_{t-1}, u_{t-2}, \dots, u_{t-\nu}) \in \{0,1\}^{\nu}, \tag{4.3}$$

共有 2^{ν} 个状态。每个时刻，状态在输入比特驱动下转移，转移边标有 n 位输出。把所有时刻的状态与转移叠画在一起，就得到网格图（trellis）。

4.2.2　从“块”到“路径”

分组码的码字是定长的，译码器可以在一个码字内部做判决；卷积码的码字是半无限序列，任一时刻的判决都与前后时刻耦合。换句话说，卷积码把“码字”换成了“路径”：编码器在网格图上走出一条路径，译码器要从接收序列中恢复这条路径。

这一视角转换是整篇论文的技术前提。它带来两个后果。第一，最优译码不再是“在 2^k 个码字里挑一个”，而是在指数多条路径里挑一条，朴素枚举不可行。第二，路径之间有大量共享前缀，可以利用动态规划把重复计算省掉——这正是下一节算法的出发点。

4.3 无记忆信道上的最大似然译码

设信道无记忆，转移概率为 $P(y_t \mid x_t)$，接收序列为 $\boldsymbol{y}_1^T = (y_1, \dots, y_T)$。最大似然序列检测（maximum likelihood sequence detection, MLSD）选择使似然函数最大的输入序列：

$$\hat{\boldsymbol{u}} = \arg\max_{\boldsymbol{u}} P\big(\boldsymbol{y}_1^T \mid \boldsymbol{x}_1^T(\boldsymbol{u})\big) = \arg\max_{\boldsymbol{u}} \sum_{t=1}^{T} \log P\big(y_t \mid x_t(\boldsymbol{u})\big). \tag{4.4}$$

把求和中的每一项定义为**分支度量**（branch metric）：

$$\lambda_t(s_{t-1}, s_t) = \log P\big(y_t \mid x_t(s_{t-1}, s_t)\big). \tag{4.5}$$

于是最大似然译码等价于在网格图上寻找累计度量最大的路径。

对加性白高斯信道（AWGN）与 BPSK，$y_t = x_t + n_t$，$n_t \sim \mathcal{N}(0, \sigma^2)$，有

$$\lambda_t(s_{t-1}, s_t) = -\frac{1}{2\sigma^2}\big\| y_t - x_t(s_{t-1}, s_t) \big\|^2 + \text{常数}, \tag{4.6}$$

最大化它就是最小化欧氏距离。对二元对称信道（BSC），交叉概率为 p，则

$$\lambda_t = \begin{cases} \log(1-p), & y_t = x_t, \\ \log p, & y_t \neq x_t, \end{cases} \tag{4.7}$$

等价于以汉明距离为度量（相差一个常数因子）。这两个式子是后来所有卷积码译码器实现的基础，但它们是教科书形式；1967 年原文使用的是概率型度量的表述，符号与归一化可能不同。

需要强调：最大似然序列检测最小化的是**整条序列**判错的概率，而不是逐比特错误概率。两者通过并集界相联系；论文的差错界针对的是比特错误概率，这一点在下一节会看得更清楚。

4.4 维特比算法：动态规划与加比选

4.4.1 递推

记 $M_t(s)$ 为从起点到时刻 t 状态 s 的所有路径中最大的累计度量，称为**路径度量**（path metric）。由动态规划的最优性原理，

$$M_t(s) = \max_{s' \in \mathcal{S}} \big[M_{t-1}(s') + \lambda_t(s', s) \big], \qquad s \in \mathcal{S}, \tag{4.8}$$

其中 $\mathcal{S}$ 是状态集，$|\mathcal{S}| = 2^{\nu}$。初始时 $M_0(0) = 0$，其余状态为 $-\infty$；若码以已知的尾比特收尾，则在终止时刻只保留终止状态。

最优性原理的证明只有一句话：若通往 s 的最优路径在 $t-1$ 时刻经过 s'，那么该路径的前缀必须是通往 s' 的最优路径；否则把前缀换成更优的一条，整条路径会更好，矛盾。因此，对每个状态只需保留一条**幸存路径**（survivor path），其余路径可以立即丢弃。

4.4.2　加比选

工程上把式 (4.8) 的三步操作合称**加比选**（add-compare-select, ACS）：

- **加**：把当前分支度量加到候选路径度量上；
- **比**：在同一状态的所有候选路径之间比较大小；
- **选**：保留最大者，并记下它的前驱状态。

每个网格截段要处理 2^{ν} 个状态，每个状态有 2^{k} 条入边；在 $k=1$ 的码率 $1/n$ 情形下，就是每状态两条入边、共 $2^{\nu+1}$ 次加比选。存储量为 2^{ν} 个路径度量，加上回溯所需的幸存比特。

译码结束时，从度量最大的终止状态出发，沿保存的前驱指针回溯，得到整条路径，再读出对应的信息比特。若不允许等到整块结束，可以用滑动窗回溯：只保留最近若干步的幸存信息，窗长通常取约束长度的数倍——这是工程经验值，不是原文的定理。

4.4.3　复杂度的意义

维特比算法的复杂度对约束长度指数增长、对序列长度线性增长，且**与信道噪声的具体实现无关**。这一点是关键。顺序译码在平均意义上更快，但它的计算量依赖噪声：遇到特别“坏”的噪声样本时，计算量会出现重尾，工程上难以给出确定的时延上界。维特比算法用固定的计算量换来了确定的译码时延，这在实时系统里往往比平均复杂度更重要。

1967 年原文把该算法描述为一种“概率型非顺序译码算法”，并没有使用“网格图”“加比选”这些后来的术语。网格图的画法与名称主要由 Forney 在 1970 年代的工作普及；Omura 在 1969 年指出了它与最大似然译码的等价性。这些都属于后续的整理与再表述，不应倒推为原文的措辞。

4.5　差错界与计算截止速率

4.5.1　转移函数与并集界

卷积码是线性码，因此可以只考虑全零路径作为参考。把所有从全零路径分出、后来又合并回全零路径的路径按重量分类：设 a_d 是汉明重量为 d 的这类路径数，w_d 是这些路径对应的信息比特错误数。定义**转移函数**（transfer function）

$$T(D,N)=\sum_{d=d_{\mathrm{free}}}^{\infty} a_d\, D^{d} N^{w_d}, \tag{4.9}$$

其中 d_{free} 是**自由距离**（free distance），即任意两条不同码字之间的最小汉明距离。对 AWGN/BPSK，把 D 取得使 D^{d} 等于距离为 d 的两条路径的成对错误概率的 Chernoff 上界；对 BSC，则取 $D=2\sqrt{p(1-p)}$。于是并集界给出

$$P_b \le \frac{1}{k}\sum_{d=d_{\mathrm{free}}}^{\infty} w_d\, a_d\, D^{d} = \frac{1}{k}\left.\frac{\partial T(D,N)}{\partial N}\right|_{N=1}. \tag{4.10}$$

对 AWGN/BPSK，$D=\exp(-E_s/N_0)=\exp(-RE_b/N_0)$，其中 E_s 是每码元能量，E_b 是每信息比特能量；对 BSC，$D=2\sqrt{p(1-p)}$。这两个 D 的表达式是标准教科书形式，原文的推导细节需对照原文。

高信噪比下，求和由自由距离项主导：

$$P_b \approx \frac{w_{d_{\text{free}}}\, a_{d_{\text{free}}}}{k}\, D^{d_{\text{free}}}. \tag{4.11}$$

因此 d_{free} 决定误码率曲线的渐近斜率，$a_{d_{\text{free}}}$ 只影响常数因子。这就是“码设计要最大化自由距离”的由来。对卷积码，d_{free} 随约束长度大致线性增长；这正是增大 K 能换来编码增益的根本原因。

4.5.2 计算截止速率

对离散无记忆信道，输入分布为 $Q(x)$、转移概率为 $P(y \mid x)$，计算截止速率定义为

$$R_0 = \max_{Q} \left[-\log_2 \sum_{y} \left(\sum_{x} Q(x) \sqrt{P(y \mid x)} \right)^2 \right]. \tag{4.12}$$

对 BSC(p)，取均匀输入，得

$$R_0 = 1 - \log_2\bigl(1 + 2\sqrt{p(1-p)}\bigr). \tag{4.13}$$

对二进制输入 AWGN（BPSK，符号能量 E_s），

$$R_0 = 1 - \log_2\bigl(1 + e^{-E_s/N_0}\bigr). \tag{4.14}$$

这两个闭式是教科书的标准结果；原文可能采用自然对数或不同的能量归一化，数值换算时需留意。

R_0 的物理含义是**顺序译码的计算截止速率**：当 $R < R_0$ 时，顺序译码每比特的平均计算量有界；当 $R > R_0$ 时，计算量的分布出现重尾，平均计算量可能发散，顺序译码在工程上不再可靠。维特比算法的计算量与噪声无关，因此恰好适合 $R > R_0$ 的区间。论文进一步证明：在 $R > R_0$ 时，式 (4.10) 的上界与原文给出的下界具有相同的指数增长率；随着 $K \to \infty$，两者的相对差距消失。这就是摘要中“指数阶紧”与“渐近最优”的含义。

这里要澄清一个容易误解的说法。“渐近最优”并不是说算法在有限 K 下最优——最大似然译码对给定码本来就是最优的；它说的是，当 K 增大时，该算法所达到的差错指数与任何译码器都无法超越的下界重合。换言之，“最优”修饰的是**差错指数**，不是某一条具体曲线。

4.5.3 为什么分界点在 R_0

在 $R < R_0$ 的区域，上下界在指数阶上不再重合，二者之间存在差距；因此论文的“渐近最优”结论只覆盖 $R > R_0$。R_0 是顺序译码的计算截止速率，也是论文中差错界由“有差距”转为“指数紧”的分界点。摘要中“除病态信道外”的限定，说明上下界重合并非对一切信道成立；具体是哪些信道、需要什么条件，需查原文的定理陈述，本章不作臆测。

4.6 与分组码的比较

摘要给出的结论是：以约束长度为自变量，最优卷积码的性能优于同等长度的分组码，且相对优势随码率增大。这句话需要放在“渐近比较”的语境里理解。

直觉上，卷积码的优势来自两点。其一，卷积码虽然约束长度有限，但码字可以很长，网格结构把“长码”的纠错能力与“局部译码”的计算方式结合了起来；同等长度的分组码要做到同样的自由距离，往往需要更大的译码表或更复杂的代数结构。其二，卷积码的距离谱更“薄”：在相同的自由距离下，低重量码字的数目 a_d 通常更少，因而并集界中的常数因子更小。

为什么优势随码率增大？一种后人的解释是：高码率下分组码的码字空间更拥挤，要同时保证高码率和大自由距离越来越困难；卷积码则可以通过增加记忆阶数、保持较小的 k/n 来维持距离增长。这个解释是教科书式的直观说明，不是原文的措辞。原文给出的是解析比较，不是一个固定的分贝增益；本章不引用具体增益数值。

反过来，卷积码也有代价：译码有延迟，存在错误传播，且复杂度随 2^{ν} 指数增长。分组码在短块、低时延场景中仍有优势。论文的比较是渐近的（K 很大），不能直接外推到 $K=3$ 或 $K=5$ 的具体设计。

4.7　结果与影响

这篇论文的影响可以分成三条线。

第一条是**算法线**。维特比算法是最大似然序列检测的动态规划实现，结构规则、便于硬件流水，1970 年代后期起随着数字集成电路的成熟，成为卷积码译码的事实标准。深空与卫星链路、GSM、IS-95、IEEE 802.11 等都采用过“卷积码 + 维特比译码”的组合。1973 年 Forney 在《Proceedings of the IEEE》上以“The Viterbi algorithm”为题的系统综述，使这一名称广为流传；此后它又被写进几乎所有数字通信教科书。

第二条是**理论线**。转移函数界、自由距离、R_0 成为编码理论的标准工具。R_0 后来成为编码理论的基本参数，在顺序译码与混合 ARQ 的吞吐分析中反复出现；它也是“信道阈值”这一直觉的早期范例——在某个速率以下，某类算法的复杂度或性能会发生质变。

第三条是**后续结构线**。1974 年 Bahl、Cocke、Jelinek、Raviv 提出逐符号最大后验概率（BCJR）算法，可以看作维特比算法在“软输出”方向上的姊妹；1993 年 Turbo 码的迭代译码，又把 BCJR 类软输入软输出译码器推到了舞台中央。维特比算法本身也没有退场：Turbo 码的简化译码器 SOVA（soft-output Viterbi algorithm）就是它的软输出版本。

从今天的视角看，1967 年论文最深刻的地方，是把“译码”从一次性的组合搜索变成了网格图上的递推。这个转变一旦发生，复杂度就从“码字数”转到了“状态数”，而状态数只与约束长度有关。后来所有基于网格的算法——包括 BCJR、Turbo 的分量译码、甚至部分 MIMO 检测——都共享这个思想。

4.8 术语对照

英文	中文
convolutional code	卷积码
constraint length	约束长度
memory order	记忆阶数
trellis	网格图
maximum likelihood sequence detection	最大似然序列检测
branch metric	分支度量
path metric	路径度量
add-compare-select (ACS)	加比选
survivor path	幸存路径
free distance	自由距离
transfer function	转移函数
union bound	并集界
computational cutoff rate	计算截止速率
sequential decoding	顺序译码
asymptotic optimality	渐近最优性

译者注

1. 原文全文未获取。本章的摘要译文依据 OpenAlex 重建的英文摘要；正文的模型、算法与界，依据 Viterbi 与 Omura 的《Principles of Digital Communication and Coding》(1979) 以及 Proakis 等教科书的标准化表述。凡原文特有的推导顺序、定理编号与数值，本章均未逐条核对。

2. 约束长度的定义有 $K = \nu$ 与 $K = \nu + 1$ 两种口径。本章取后者，即“输出所依赖的输入比特数”。引用具体码参数（例如“$K = 7$”）时，务必确认原文用的是哪一种。

3. “网格图”“加比选”“维特比算法”这些术语在 1967 年原文中并未以今天的形式出现。1967 年论文描述的是“概率型非顺序译码算法”；网格图的图形化与命名主要由 Forney 在 1970 年代普及，Omura（1969）指出了它与最大似然译码的等价性。

4. 式 (4.6)、(4.10) 中的分支度量与并集界是教科书形式。原文可能以概率、似然或“距离”的不同归一化写出；不同教材对 E_s、E_b、N_0 的单双边约定也不同，代入数值时需核对。

5. “渐近最优”指的是差错指数随 $K \to \infty$ 达到下界，不是指有限 K 下的逐点最优。注意方向：论文的结论是 $R > R_0$ 时渐近最优；$R < R_0$ 是顺序译码计算有界的区间，两者不要记反。$R < R_0$ 时下界本身不紧，不能据此下结论。

6. 摘要中的“同等长度的分组码”是一个需要对照原文确认的比较口径：是以相同的约束长度、相同的译码跨度，还是相同的码字数作比较，会直接影响结论的强度。本章只复述摘要的定性结论，不引用具体分贝数。

7. 自由距离随约束长度“大致线性增长”是定性说法；不同码率、不同生成多项式下斜率不同，具体数值需查码表或原文。